

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 «Программная инженерия» –
Системное и прикладное программное обеспечение

ОТЧЁТ

по дисциплине «Разработка мобильных приложений»

Мобильное приложение для трейдинга и инвестиций

Выполнили: студенты

Покалюхин Илья Игоревич

Нестеров Владислав Алексеевич

Джохадзе Анна Бекаевна

Бондаренко Артём Андреевич

Лашкул Андрей Владимирович

Проверил:

Ключев Аркадий Олегович

Санкт-Петербург, 2026 г.

СОДЕРЖАНИЕ

Введение	4
1 Техническое задание	6
1.1 Введение	6
1.2 Назначение разработки	6
1.3 Заинтересованные лица и их требования	6
1.4 Требования к программе или программному изделию	7
1.4.1 Функции системы	7
1.4.2 Состав системы	8
1.4.3 Характеристики системы	9
1.5 Оценка сроков, бюджета и возможностей разработчиков	10
2 Архитектура системы	12
2.1 Структура системы и потоки данных	12
2.2 Обоснование выбора технологий	13
2.3 Схема базы данных	13
3 Описание реализации	14
3.1 Драйвер ядра	14
3.2 Сервис котировок	14
3.3 Data-сервис	14
3.4 Gateway	14
3.5 Android-приложение	15
3.6 Ключевые технические решения	15
3.7 Пользовательский интерфейс	15
4 Тестирование	18
4.1 Юнит- и интеграционные тесты	18
4.2 Нагрузочное тестирование 10 000 клиентов	18
Заключение	19
Список литературы	20
Приложение А. Формат записи драйвера и команды ioctl	21
Приложение Б. Схема базы данных	22
Приложение В. Публичный API Gateway	25

Приложение Г. Протокол нагрузочного тестирования	26
--	----

ВВЕДЕНИЕ

Настоящий документ описывает разработку мобильного приложения для трейдинга и инвестиций вместе с обслуживающей его серверной экосистемой. Введение позволяет читателю быстро понять назначение работы: ниже обоснована её актуальность, сформулированы цель и задачи.

Актуальность. Розничные инвестиции переживают период бурного роста: число частных инвесторов на Московской бирже превысило 30 млн человек, а доля сделок физических лиц на рынке акций устойчиво превышает 70 %. Подавляющее большинство таких инвесторов взаимодействует с рынком исключительно через мобильные приложения — для них смартфон фактически заменил биржевой терминал. Это формирует жёсткие требования к мобильным брокерским системам: котировки должны доставляться в реальном времени, торговые операции — исполняться корректно и атомарно, а серверная инфраструктура — выдерживать одновременную работу десятков тысяч клиентов в моменты рыночной волатильности, когда нагрузка возрастает кратно. Разработка такой системы охватывает весь стек технологий — от низкоуровневого взаимодействия с источником данных до нативного мобильного клиента и наблюдаемой распределённой серверной части, — что делает её показательной задачей, затрагивающей ключевые аспекты современной мобильной и backend-разработки. Таким образом, создание масштабируемого мобильного биржевого терминала является своевременной и практически значимой задачей.

Цель работы — спроектировать и реализовать полную экосистему биржевого терминала: от источника котировок на уровне ядра операционной системы до нативного мобильного клиента, способную обслуживать 10 000 одновременных клиентов.

Задачи работы:

1. реализовать драйвер ядра Linux, имитирующий поток биржевых котировок;
2. реализовать микросервис сбора котировок с публикацией в брокер сообщений и записью истории в аналитическую СУБД;
3. реализовать сервис работы с базой данных: счета, ордера, сделки, портфели, матчинг лимитных заявок;

4. реализовать gateway-сервис для мобильных клиентов: аутентификация, потоковая раздача котировок, проксирование торговых операций;
5. разработать нативное Android-приложение (Kotlin, Jetpack Compose);
6. провести нагрузочное тестирование на 10 000 одновременных клиентов;
7. обеспечить наблюдаемость системы (распределённые трейсы, метрики, логи).

1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ

1.1 Введение

Настоящее техническое задание (ТЗ) определяет назначение, состав, функции и характеристики мобильного биржевого терминала, а также требования к нему со стороны заинтересованных лиц. Документ служит основанием для проектирования, реализации и приёмки системы.

ТЗ составлено по результатам анализа предметной области по следующей методике: определение круга заинтересованных лиц; выделение значимых аспектов разработки (функциональность, инструментальная база, надёжность, безопасность, энергопотребление); сбор требований каждой из сторон; оценка сроков, бюджета и возможностей команды; формулирование заведомо выполнимого ТЗ, удовлетворяющего заказчика. Область применения — учебная разработка экосистемы биржевого терминала, демонстрирующая полный цикл от источника котировок до мобильного клиента.

1.2 Назначение разработки

Функциональное назначение. Система предназначена для предоставления частным инвесторам доступа к биржевым торгам с мобильного устройства: просмотр котировок в реальном времени, анализ истории цен, выставление и отмена торговых заявок, ведение портфеля и истории операций.

Эксплуатационное назначение. Серверная часть функционирует в виде набора контейнеризированных микросервисов в среде Linux/Docker и рассчитана на одновременное обслуживание до 10 000 клиентов; клиентская часть — нативное Android-приложение (и опциональный кросс-платформенный клиент на React Native), работающее на пользовательских смартфонах.

1.3 Заинтересованные лица и их требования

Перед формулированием требований определён круг заинтересованных лиц — сторон, влияющих на систему или зависящих от неё. Для каждой стороны выделены ключевые требования (Таблица 1), которые легли в основу функциональных и нефункциональных требований к системе.

Таблица 1 — Заинтересованные лица и их требования

Заинтересованное лицо	Ключевые требования
Частный инвестор (пользователь)	Котировки в реальном времени; быстрый и понятный интерфейс; мгновенное исполнение рыночных заявок; корректный учёт портфеля и P&L; история операций; экономия заряда батареи и трафика
Брокерская компания (заказчик)	Корректность и аудируемость торговых операций; обслуживание десятков тысяч клиентов; защита средств клиентов; низкая стоимость владения; возможность развития функционала
Поставщик котировок (биржа)	Надёжный приём потока тиков без потерь; ограничение нагрузки на источник; корректная ретрансляция данных
Разработчик мобильного клиента	Стабильный документированный API; единый язык со стеклом (Kotlin); тестируемость интерфейса без эмулятора
Разработчик серверной части	Чёткое разделение ответственности сервисов; параметризованный доступ к БД; воспроизводимое окружение
Инженер по эксплуатации (DevOps/SRE)	Контейнеризация и оркестрация; метрики, логи, распределённые трейсы; устойчивость к перегрузке и сбоям отдельных сервисов
Специалист по ИБ и комплаенсу	Аутентификация и авторизация; хранение паролей в виде хэшей; защита от инъекций и перебора; разделение публичного и внутреннего API
Инженер по тестированию (QA)	Детерминированные тесты бизнес-логики; нагрузочное тестирование; скриншот-тестирование интерфейса
Служба технической поддержки	Прослеживаемость операций по сквозному идентификатору; понятные коды ошибок

1.4 Требования к программе или программному изделию

1.4.1 Функции системы

Система должна реализовывать следующие функции:

- регистрация и аутентификация пользователей (выдача и обновление токенов доступа);
- просмотр списка торговых инструментов с текущей ценой и изменением за день;
- потоковая доставка котировок в реальном времени с подпиской по выбранным символам;
- просмотр истории цен в виде свечей с интервалами 1m, 5m, 15m, 1h, 1d;
- выставление рыночных (MARKET) и лимитных (LIMIT) заявок на покупку и продажу;
- автоматическое исполнение лимитных заявок при достижении указанной цены;
- отмена активных заявок;
- ведение денежного счёта и позиций, расчёт средней цены, стоимости портфеля и нереализованной прибыли/убытка (P&L);
- просмотр истории заявок и сделок;
- идемпотентность торговых операций (повторный запрос с тем же идентификатором не создаёт дубликат);
- сбор и долговременное хранение истории котировок для аналитики.

1.4.2 Состав системы

Система строится как набор слабосвязанных компонентов; их перечень, технологии и назначение приведены ниже (Таблица 2). Нумерация компонентов (#1...#7) используется далее в тексте и на схеме архитектуры.

Таблица 2 — Состав системы

Компонент	Технологии и назначение
#7 Драйвер-имитатор	C, модуль ядра Linux; генерация потока биржевых тиков
#6 Сервис котировок	Go; чтение тиков, публикация в Redis, запись истории в ClickHouse
#4 Data-сервис	Kotlin/Ktor + PostgreSQL; счета, ордера, сделки, матчинг лимиток
#3 Gateway	Kotlin/Ktor; аутентификация, WebSocket-котировки, проксирование
#1 Android-приложение	Kotlin/Jetpack Compose; основной мобильный клиент
#2 RN-клиент (опц.)	React Native/Expo; кросс-платформенный клиент
#5 Нагрузочный имитатор	Go; проверка обслуживания 10 000 клиентов
Инфраструктура	Redis, ClickHouse, PostgreSQL, OpenTelemetry/Jaeger, Docker Compose

1.4.3 Характеристики системы

К системе предъявляются следующие значимые характеристики, отражающие важные аспекты разработки:

- **Производительность.** Обслуживание до 10 000 одновременных клиентов; целевая задержка ответа $p99 \leq 250$ мс; доставка котировок в реальном времени.
- **Надёжность.** Доля ошибочных запросов менее 0,1 %; отказ одного сервиса не останавливает остальные (развязка через брокер сообщений); атомарность торговых транзакций (баланс не может уйти в минус).
- **Безопасность.** Аутентификация по JWT; хранение паролей в виде bcrypt-хэшей; только параметризованные SQL-запросы (защита от инъекций); ограничение частоты запросов (rate limiting); разделение публичного и внутреннего API.
- **Энергопотребление (мобильный аспект).** Доставка котировок выполняется push-моделью по одному WebSocket-соединению вместо периодического опроса; интерфейс перерисовывает только изменившиеся элементы (StateFlow); объём трафика минимизируется (компактные сообщения, дельта-обновления).

- **Инструментальный аспект.** Единый язык Kotlin для серверной части и Android-клиента; контейнеризация (Docker Compose); сборка Gradle/Go; наблюдаемость на основе OpenTelemetry.
- **Прикладной аспект.** Покрытие полного пользовательского сценария инвестора — от регистрации до сделки и анализа портфеля.
- **Масштабируемость.** Stateless-Gateway, допускающий горизонтальное масштабирование; колоночное хранилище истории котировок.
- **Сопровождаемость.** Сквозные распределённые трейсы, метрики, структурированные логи, документация (Dokka, Obsidian).

1.5 Оценка сроков, бюджета и возможностей разработчиков

Возможности команды. Разработку ведёт команда из пяти студентов направления «Программная инженерия», владеющих языками Kotlin, Go, C, SQL и веб-технологиями; зоны ответственности распределены по компонентам системы.

Сроки. Срок разработки — один учебный семестр (≈4 месяца); работа разбита на этапы (Таблица 3) с итеративной сдачей результатов.

Таблица 3 — Этапы разработки

Этап	Содержание	Результат
1	Драйвер ядра	Символьные устройства, поток тиков
2	Сбор котировок	quotes-go: Redis Pub/Sub, история в ClickHouse
3	Data-сервис	Счета, ордера, сделки, матчинг лимиток
4	Gateway	Аутентификация, WebSocket, проксирование
5	Android-клиент	Экраны, графики, формы ордеров
6	Нагрузочное тестирование	Подтверждение 10 000 клиентов
7	Наблюдаемость и отчёт	Трейсы, метрики, документация

Бюджет. Денежный бюджет нулевой (учебный проект): используются только свободное и открытое ПО (Linux, PostgreSQL, Redis, ClickHouse, Ktor, OpenTelemetry, Jaeger) и собственная вычислительная техника; инфраструктура запускается локально в Docker.

С учётом перечисленных возможностей и ограничений ТЗ сформулировано как заведомо выполнимое: объём функций соответствует одному

семестру, выбранные технологии знакомы команде, а целевая нагрузка (10 000 клиентов) проверяется программным имитатором без аренды облачных ресурсов. Это удовлетворяет основного заказчика — преподавателя дисциплины — и моделирует реальную постановку задачи со стороны брокерской компании.

2 АРХИТЕКТУРА СИСТЕМЫ

Современные брокерские приложения (Тинькофф Инвестиции, СберИнвестор, Robinhood) построены по схожей схеме: тонкий мобильный клиент, единая точка входа (API-gateway), потоковая доставка котировок по WebSocket и отдельные сервисы исполнения ордеров и учёта позиций; историческое хранение рыночных данных вынесено в колоночные СУБД, оптимизированные под время-ряды. Архитектура разрабатываемой системы следует этой проверенной схеме.

2.1 Структура системы и потоки данных

Система состоит из семи основных компонентов (Рисунок 1); их состав приведён ранее (Таблица 2). Драйвер ядра (#7) генерирует тики; Go-сервис (#6) читает их, публикует в Redis и пишет в ClickHouse; Data-сервис (#4) ведёт счета и исполняет ордера; Gateway (#3) обслуживает клиентов — Android-приложение (#1) и нагрузочный имитатор (#5).

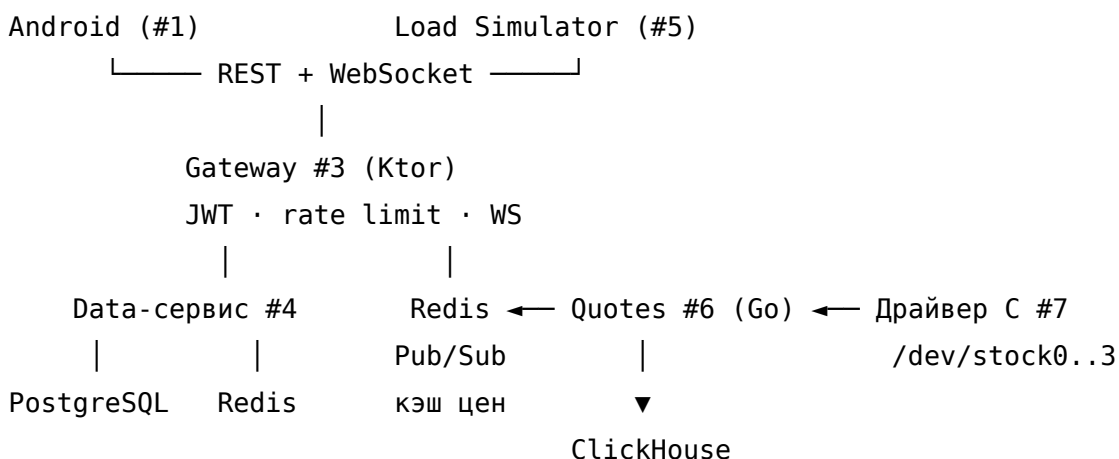


Рисунок 1 — Архитектура системы

Поток котировок: драйвер → quotes-go → Redis Pub/Sub (quotes:ticks) + ClickHouse (батчи) → Gateway (одна подписка, fan-out по WebSocket) → клиенты.

Поток торговли: клиент → Gateway (JWT, userId из токена) → Data-сервис: market исполняется по цене из Redis-кэша; limit матчится фоновым воркером по потоку тиков. Сделка, позиция и баланс изменяются в одной транзакции.

2.2 Обоснование выбора технологий

Выбор технологий для каждого слоя системы и его обоснование приведены ниже (Таблица 4).

Таблица 4 — Обоснование выбора технологий

Слой	Выбор и обоснование
Драйвер	C, модуль ядра Linux — имитация источника котировок на максимально низком уровне; модель геометрического броуновского движения
Сбор котировок	Go — дешёвые горутины для блокирующего чтения символьных устройств, простой HTTP/SSE
Бэкенд	Kotlin + Ktor — корутины, единый язык с Android-клиентом; голый SQL без ORM (требование задания)
Брокер/кэш	Redis — Pub/Sub для тиков и кэш последних цен
СУБД	PostgreSQL — транзакционная целостность счетов и сделок
История	ClickHouse — колоночное хранение тиков, свечи через материализованные представления
Клиент	Kotlin + Jetpack Compose, MVVM, StateFlow
Наблюдаемость	OpenTelemetry + Jaeger — сквозные трейсы

2.3 Схема базы данных

Оперативные данные (счета, ордера, сделки, позиции) хранятся в PostgreSQL, история котировок — в ClickHouse. Логическая структура:

```
users      (id, login UNIQUE, password_hash, created_at)
accounts   (id, user_id FK, currency, cash_balance CHECK >= 0)
instruments(id, symbol UNIQUE, name, dev_idx)
orders     (id, user_id, symbol, side, type, qty, price, filled_qty,
            status, client_order_id UNIQUE, created_at, updated_at)
trades     (id, order_id, user_id, symbol, side, qty, price, fee,
            executed_at)
positions  (user_id, symbol, qty CHECK >= 0, avg_price, PK(user_id, symbol))
```

ClickHouse: ticks (symbol, ts, price, volume, seq) ENGINE=MergeTree,
свечи candles_1m — AggregatingMergeTree, заполняется материализованным представлением. Полный DDL приведён в приложении Б.

3 ОПИСАНИЕ РЕАЛИЗАЦИИ

3.1 Драйвер ядра

Модуль `stock_gen.c` создаёт символьные устройства `/dev/stock0..3`. Цена моделируется геометрическим броуновским движением. Запись — 32 байта (little-endian): `seq u64, timestamp_ns s64, price_udollar s64, volume u32, dev_idx u32`. Поддерживаются блокирующее чтение, `poll` и `ioctl` (установка базовой цены, волатильности, дрефта, интервала тика). Формат записи и команды `ioctl` подробно описаны в приложении А.

3.2 Сервис котировок

Go-сервис читает устройства (или генерирует тики встроенным GBM-симулятором, когда `/dev/stock*` недоступны), маппит индекс устройства на тикер (SBER, GAZP, AAPL, BTC) и раздаёт данные потребителям: REST/SSE для отладки, Redis Pub/Sub `quotes:ticks` + ключи `quote:last:<symbol>` для остальных сервисов, батч-запись в ClickHouse (раз в секунду или по 5000 строк). Метрики (тиков/сек, ошибки записи) экспонируются в формате Prometheus.

3.3 Data-сервис

Ядро бизнес-логики на Ktor. Работа с PostgreSQL — голый SQL через JDBC + HikariCP: репозитории принимают `Connection`, поэтому операции компонуются в одну транзакцию. Только параметризованные запросы.

Исполнение ордера: блокировка строки счёта (`SELECT ... FOR UPDATE`), проверка средств или позиции, запись сделки, пересчёт средней цены позиции и баланса — атомарно. Недостаток средств — статус `REJECTED`. Идемпотентность обеспечивает уникальный `client_order_id`. Лимитные заявки исполняет фоновый воркер, слушающий `quotes:ticks`: пересечённые лимитки выбираются запросом с `FOR UPDATE SKIP LOCKED` и исполняются по цене тика.

3.4 Gateway

Единая точка входа: регистрация и вход (bcrypt + JWT HS256, access 15 мин / refresh 7 дней), список котировок с изменением за день, история свечей из ClickHouse (агрегация тиков на лету), WebSocket-стрим с подпиской по символам (одна подписка на Redis на весь процесс, fan-out через SharedFlow),

проксирование торговых маршрутов в Data-сервис с подстановкой `userId` из JWT и rate limiting на IP. Публичный контракт API приведён в приложении В.

3.5 Android-приложение

Kotlin + Jetpack Compose, архитектура MVVM (ViewModel + StateFlow), ручной DI. Экраны: авторизация, рынок (живые цены по WebSocket), карточка инструмента с графиком (Canvas) и формой ордера Market/Limit, портфель с P&L, история ордеров и сделок с отменой активных лимиток. JWT хранится в DataStore; при 401 клиент автоматически обновляет access-токен и повторяет запрос.

3.6 Ключевые технические решения

Помимо описанных выше компонентов, в системе приняты следующие сквозные технологические решения:

- **Redis Pub/Sub как шина тиков** развязывает генерацию котировок и потребителей: Gateway и Data-сервис подписаны независимо, отказ одного не влияет на другой.
- **Денежные поля в JSON передаются строками** — `NUMERIC(18,6)` не представим в `double` без потери точности.
- **Сквозной трейсинг**: Gateway инжектирует W3C `traceparent` в запросы к Data-сервису; спаны SQL-транзакций становятся детьми клиентского запроса. Контекст переносится между корутинами через `opentelemetry-extension-kotlin`.
- **Защита от перегрузки**: rate limiting в Gateway, неблокирующие буферы в каналах тиков (медленный потребитель теряет старые тики, но не останавливает систему).

3.7 Пользовательский интерфейс

Внешний вид экранов Android-приложения показан ниже (Рисунок 2, Рисунок 3). Изображения получены скриншот-тестами (см. раздел 4).

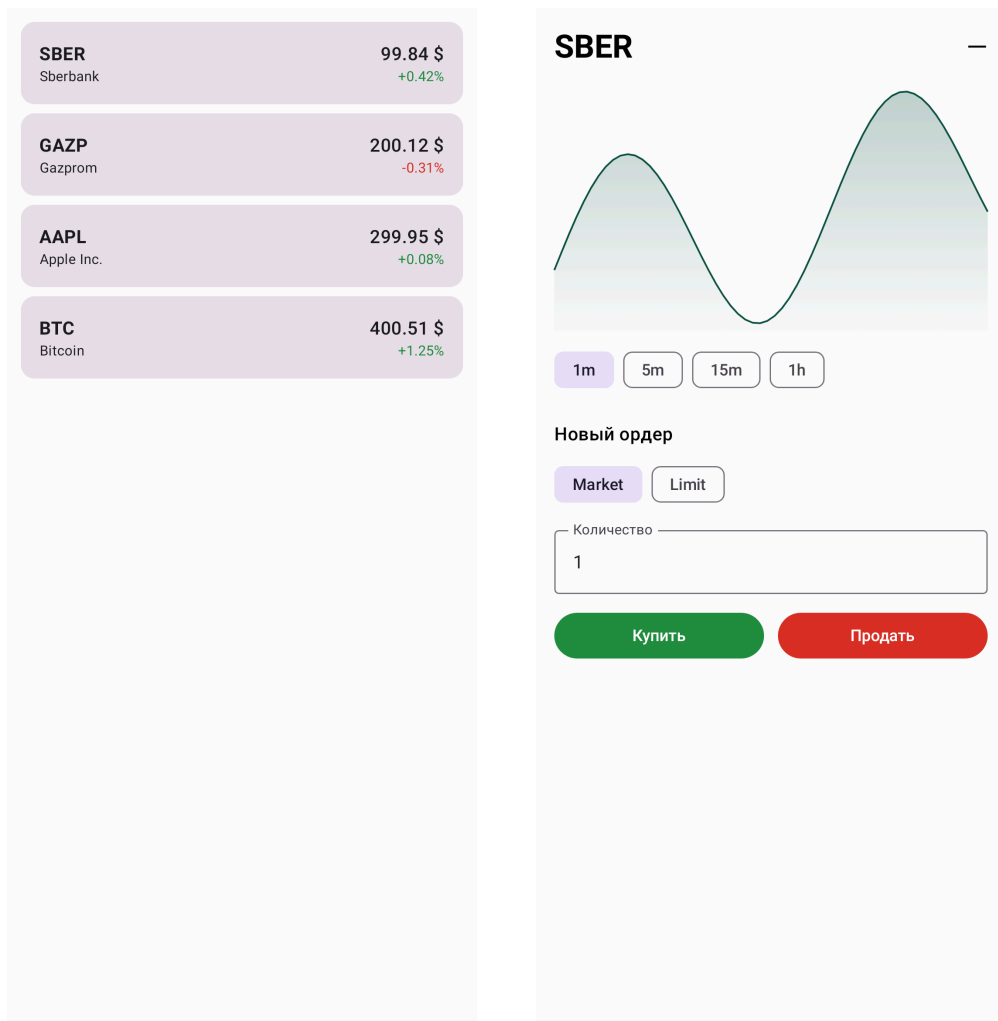


Рисунок 2 — Экраны «Рынок» и «Карточка инструмента»

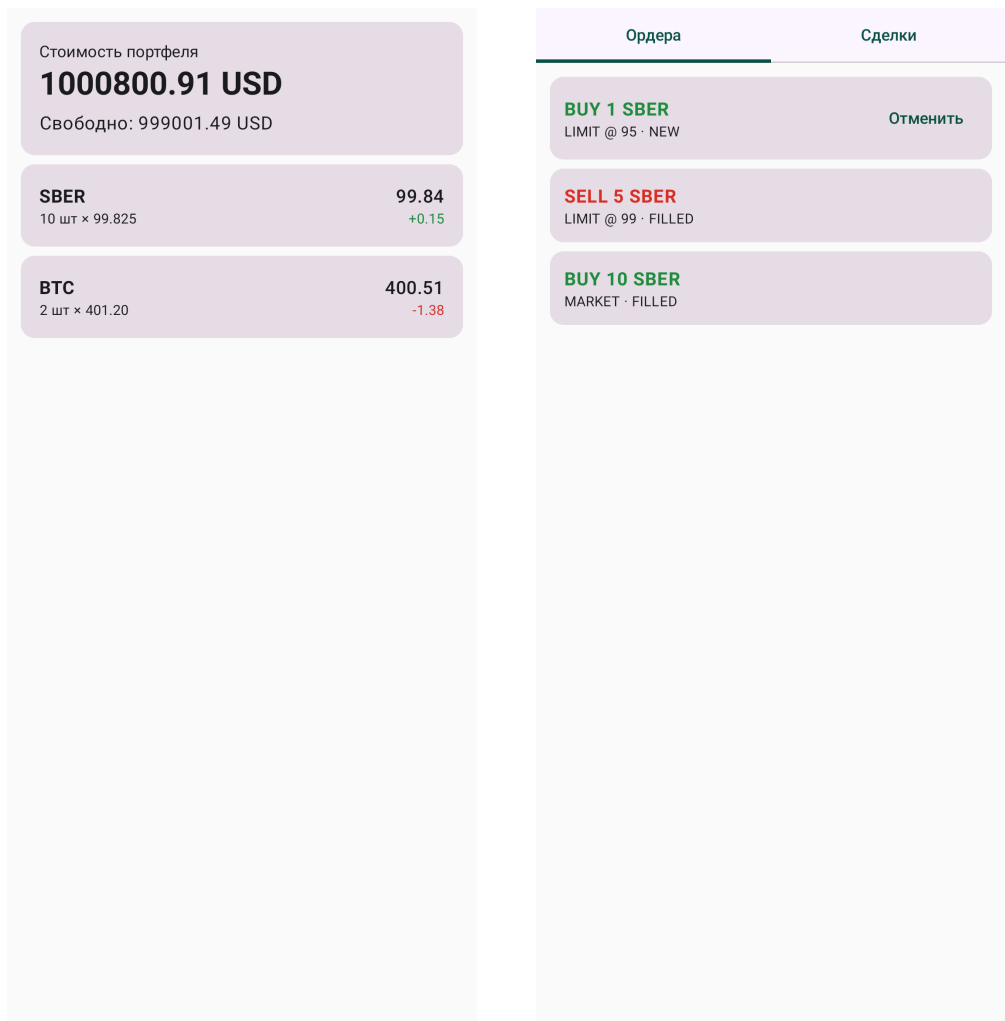


Рисунок 3 — Экраны «Портфель» и «История ордеров»

4 ТЕСТИРОВАНИЕ

4.1 Юнит- и интеграционные тесты

В каждом сервисе — юнит-тесты бизнес-логики (маппинг символов, кодирование батчей, средняя цена позиции, пересечение лимиток, JWT, перцентили). UI Android-приложения покрыт скриншот-тестами (Robolectric Native Graphics + Roborazzi): экраны рендерятся без эмулятора, изображения из этих тестов приведены на рисунках выше. Интеграционный сценарий проверен на реальном окружении (PostgreSQL, Redis, ClickHouse): регистрация → market-ордер → сделка → портфель; лимитная заявка исполняется воркером при достижении цены; повторный запрос с тем же `clientId` возвращает существующий ордер.

4.2 Нагрузочное тестирование 10 000 клиентов

Имитатор поднимает 10 000 виртуальных клиентов (горутин), каждый регистрируется и выполняет случайные действия с заданными весами; 1 000 клиентов дополнительно держат WebSocket-подписку. Плавный ramp-up 60 с, рабочая фаза 90 с, 2 000 запросов/с. Результаты по типам действий приведены ниже (Таблица 5); полный протокол теста — в приложении Г.

Таблица 5 — Результаты нагрузочного теста на 10 000 клиентов

Действие	Запросов	Ошибок	p50, мс	p95, мс	p99, мс	max, мс
quotes	110 119	0	10.8	71.9	120.0	316.6
portfolio	49 257	0	11.9	73.9	122.9	317.9
order	36 987	5	22.9	91.3	143.6	322.9
trades	24 543	0	11.1	71.5	119.6	264.1
history	24 277	5	30.4	126.9	209.4	506.2
register	10 000	0	12.6	52.2	121.9	402.2

Итого 255 183 запроса, 10 ошибок (0,004 %). WebSocket: 1 000 стабильных соединений, 6,08 млн доставленных тиков, ни одного переподключения.

ЗАКЛЮЧЕНИЕ

В ходе работы реализована полная экосистема биржевого терминала:

1. драйвер ядра Linux с имитацией котировок (GBM);
2. Go-сервис сбора котировок с Redis Pub/Sub и записью в ClickHouse;
3. Data-сервис на Ktor с голым SQL: счета, ордера, сделки, матчинг лимиток;
4. Gateway с JWT-аутентификацией, WebSocket-стримом и rate limiting;
5. Android-приложение на Jetpack Compose (MVVM) и опциональный кросс-платформенный клиент на React Native;
6. нагрузочный имитатор, подтвердивший обслуживание 10 000 одновременных клиентов с $p_{99} \leq 210$ мс и долей ошибок 0,004 %;
7. наблюдаемость: сквозные трейсы OpenTelemetry, метрики, структурированные логи; документация Dokka и Obsidian.

Все задачи, поставленные во введении, выполнены, цель работы достигнута. Направления развития: частичное исполнение ордеров (PARTIALLY_FILLED), стакан заявок и матчинг «клиент против клиента», горизонтальное масштабирование Gateway за балансировщиком.

СПИСОК ЛИТЕРАТУРЫ

1. Love R. Linux Kernel Development. — 3-е изд. — Addison-Wesley, 2010.
2. Donovan A., Kernighan B. The Go Programming Language. — Addison-Wesley, 2015.
3. Ktor Documentation. — URL: <https://ktor.io/docs/> (дата обращения: 12.06.2026).
4. Jetpack Compose Documentation. — URL: <https://developer.android.com/compose> (дата обращения: 12.06.2026).
5. Redis Pub/Sub. — URL: <https://redis.io/docs/latest/develop/interact/pubsub/> (дата обращения: 12.06.2026).
6. ClickHouse Documentation. — URL: <https://clickhouse.com/docs> (дата обращения: 12.06.2026).
7. OpenTelemetry Specification. — URL: <https://opentelemetry.io/docs/specs/otel/> (дата обращения: 12.06.2026).
8. Hull J. Options, Futures, and Other Derivatives. — 11-е изд. — Pearson, 2021.

ПРИЛОЖЕНИЕ А. ФОРМАТ ЗАПИСИ ДРАЙВЕРА И КОМАНДЫ IOCTL

Каждое символьное устройство `/dev/stock0..3` отдаёт поток записей фиксированного размера 32 байта в порядке `little-endian`. Структура записи приведена ниже (Таблица 6).

Таблица 6 — Формат записи драйвера (32 байта, `little-endian`)

Поле	Тип	Байт	Описание
<code>seq</code>	<code>u64</code>	8	порядковый номер тика устройства
<code>timestamp_ns</code>	<code>s64</code>	8	время генерации, наносекунды
<code>price_udollar</code>	<code>s64</code>	8	цена в микродолларах (10^{-6} \$)
<code>volume</code>	<code>u32</code>	4	объём тика
<code>dev_idx</code>	<code>u32</code>	4	индекс устройства (0...3)

Через интерфейс `ioctl` настраиваются параметры генерации для каждого устройства: базовая цена, волатильность, дрейф (снос) и интервал между тиками. Индекс устройства сопоставляется тикеру: 0 → SBER, 1 → GAZP, 2 → AAPL, 3 → BTC.

ПРИЛОЖЕНИЕ Б. СХЕМА БАЗЫ ДАННЫХ

DDL оперативной БД (PostgreSQL):

```
CREATE TABLE users (  
    id          BIGSERIAL PRIMARY KEY,  
    login       TEXT          NOT NULL UNIQUE,  
    password_hash TEXT        NOT NULL,  
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE accounts (  
    id          BIGSERIAL PRIMARY KEY,  
    user_id     BIGINT        NOT NULL REFERENCES users (id),  
    currency    TEXT          NOT NULL DEFAULT 'USD',  
    cash_balance NUMERIC(18,2) NOT NULL DEFAULT 0 CHECK (cash_balance >= 0),  
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE UNIQUE INDEX accounts_user_currency_uq ON accounts (user_id,  
currency);  
  
CREATE TABLE instruments (  
    id          BIGSERIAL PRIMARY KEY,  
    symbol      TEXT NOT NULL UNIQUE,  
    name        TEXT NOT NULL,  
    dev_idx     INT  NOT NULL  
);  
  
CREATE TABLE orders (  
    id          BIGSERIAL PRIMARY KEY,  
    user_id     BIGINT        NOT NULL REFERENCES users (id),  
    symbol      TEXT          NOT NULL,  
    side        TEXT          NOT NULL CHECK (side IN ('BUY', 'SELL')),  
    type        TEXT          NOT NULL CHECK (type IN ('MARKET', 'LIMIT')),  
    qty         NUMERIC(18,6) NOT NULL CHECK (qty > 0),  
    price       NUMERIC(18,6),  
    filled_qty  NUMERIC(18,6) NOT NULL DEFAULT 0,  
    status      TEXT          NOT NULL DEFAULT 'NEW' CHECK (status IN (  
        'NEW', 'FILLED', 'PARTIALLY_FILLED', 'CANCELLED', 'REJECTED')),  
    client_order_id TEXT UNIQUE,  
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
```

```

);
CREATE INDEX orders_status_symbol_idx ON orders (status, symbol);
CREATE INDEX orders_user_idx ON orders (user_id, created_at DESC);

CREATE TABLE trades (
    id          BIGSERIAL PRIMARY KEY,
    order_id    BIGINT      NOT NULL REFERENCES orders (id),
    user_id     BIGINT      NOT NULL REFERENCES users (id),
    symbol      TEXT        NOT NULL,
    side        TEXT        NOT NULL CHECK (side IN ('BUY', 'SELL')),
    qty         NUMERIC(18,6) NOT NULL,
    price       NUMERIC(18,6) NOT NULL,
    fee         NUMERIC(18,6) NOT NULL DEFAULT 0,
    executed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX trades_user_executed_idx ON trades (user_id, executed_at DESC);

CREATE TABLE positions (
    user_id     BIGINT      NOT NULL REFERENCES users (id),
    symbol      TEXT        NOT NULL,
    qty         NUMERIC(18,6) NOT NULL CHECK (qty >= 0),
    avg_price   NUMERIC(18,6) NOT NULL,
    PRIMARY KEY (user_id, symbol)
);

```

DDL аналитического хранилища (ClickHouse): сырые тики и материализованные свечи 1m.

```

CREATE TABLE trading.ticks (
    symbol LowCardinality(String),
    ts     DateTime64(3),
    price  Decimal(18, 6),
    volume UInt32,
    seq    UInt64
) ENGINE = MergeTree ORDER BY (symbol, ts);

CREATE TABLE trading.candles_1m (
    symbol LowCardinality(String),
    t      DateTime,
    open   AggregateFunction(argMin, Decimal(18, 6), DateTime64(3)),
    high   SimpleAggregateFunction(max, Decimal(18, 6)),
    low    SimpleAggregateFunction(min, Decimal(18, 6)),

```

```
close AggregateFunction(argMax, Decimal(18, 6), DateTime64(3)),  
volume SimpleAggregateFunction(sum, UInt64)  
) ENGINE = AggregatingMergeTree ORDER BY (symbol, t);
```


ПРИЛОЖЕНИЕ В. ПУБЛИЧНЫЙ API GATEWAY

Публичный контракт API (Gateway, порт 8080). Все защищённые маршруты требуют заголовок `Authorization: Bearer <accessToken>`. Денежные поля передаются строками.

Аутентификация

POST /auth/register { login, password } → 201 { userId, accessToken, refreshToken }

POST /auth/login { login, password } → 200 { accessToken, refreshToken } | 401

POST /auth/refresh { refreshToken } → 200 { accessToken } | 401

Котировки

GET /quotes → [{ symbol, name, last, changePct, volume }]

GET /quotes/{symbol}/history?interval&from&to&limit
→ [{ t, open, high, low, close, volume }]

GET /ws?token=<jwt> WebSocket

→ { "action": "subscribe", "symbols": ["SBER"] } | "unsubscribe"

← { symbol, price, ts, volume, seq }

Торговля

GET /portfolio → { cash, positions[], totalValue }

GET /orders?status=&limit= → [Order]

POST /orders { symbol, side, type, qty, price?, clientId? } → 201

Order

DELETE /orders/{id} → Order (CANCELLED)

GET /trades?limit= → [Trade]

Коды ошибок: 401 — недействительный токен, 422 — ошибка валидации, 429 — превышение лимита частоты запросов.

ПРИЛОЖЕНИЕ Г. ПРОТОКОЛ НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ

Конфигурация запуска: 10 000 клиентов, из них 1 000 с WebSocket-подпиской; плавный ramp-up 60 с, рабочая фаза 90 с, интервал действий 5 с; целевой Gateway — `http://localhost:8080`. Идентификатор запуска `run1781299613`, завершён 2026-06-12T21:29:28Z.

Сводные показатели:

Таблица 7 — Сводные показатели нагрузочного теста

Показатель	Значение
Всего запросов	255 183
Всего ошибок	10
Доля ошибок	0,004 %
WebSocket-соединений	1 000
Отказов подключения WebSocket	0
Доставлено тиков по WebSocket	6 079 500

Детализация по типам действий (количество запросов, ошибки и перцентили задержки) приведена ранее (Таблица 5). Сырой отчёт сохранён в файле `load/load-test-10k.json`.